



COMPUTER ARCHITECTURE

Hamid Sarbazi-Azad

Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

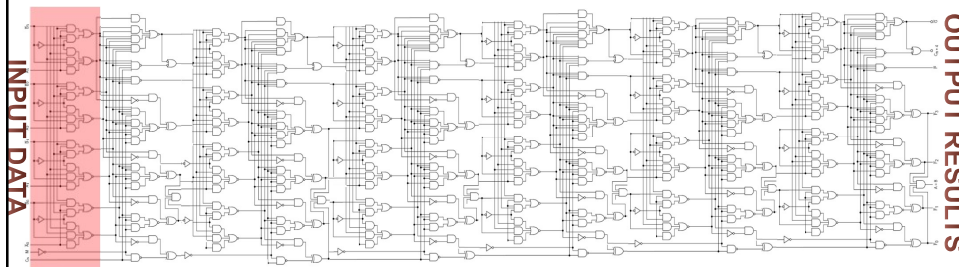
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

2

Pipelining

Why pipelining?



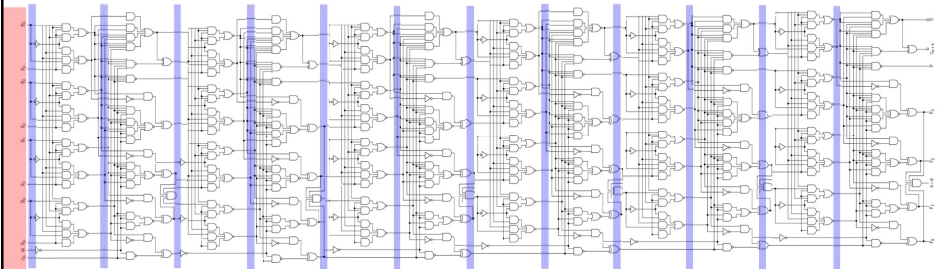
From the time INPUT DATA is given until OUTPUT RESULTS are ready, at any time, some **limited number of gates at a certain level** are activated and required to operate.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

3

Pipelining



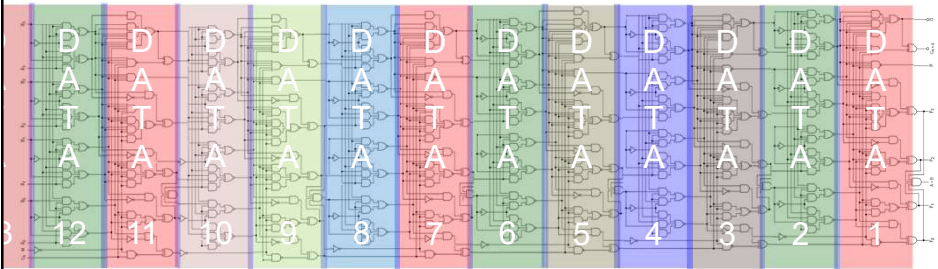
Many INPUT DATA sets can be processed in pipelined fashion.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

4

Pipelining



Many INPUT DATA sets can be processed in pipelined fashion.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

5

Pipelining

Many INPUT DATA sets can be processed in pipelined fashion.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

6

Pipelining

Processing N inputs takes $N \times \tau$ seconds.

Processing N inputs takes $\tau + (N - 1) \times \frac{\tau}{k}$ seconds.

MIPS Possible Implementations

- **Single-cycle** implementation:
 - Update all state elements (**PC**, register file, data memory) at the end of a clock cycle
- **Multi-cycle** implementation:
 - Update different state elements (**PC**, register file, data memory) at the end of different clocks
- **Pipelined** implementation:
 - One cycle per pipeline stage
 - Data required for each stage needs to be stored separately

MIPS Stages

- **Five** Stages
 - **IF**: Instruction Fetch
 - **ID**: Instruction Decode and Register Read
 - **EX**: Execute an operation or calculate an address
 - **MEM**: Access an operand in data memory
 - **WB**: Write back the result into a register
- **Idea**:
 - **Each stage takes 1 clock cycle**
 - General flow of data is from one stage to the next

MIPS Stages

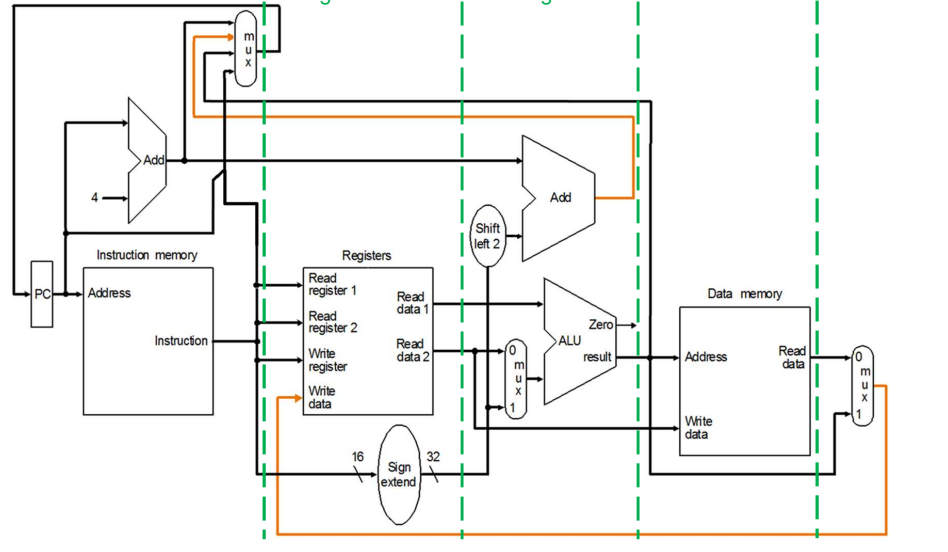
IF: Instruction Fetch

ID: Instruction decoded
Register file read

EX: Execute/
address generation

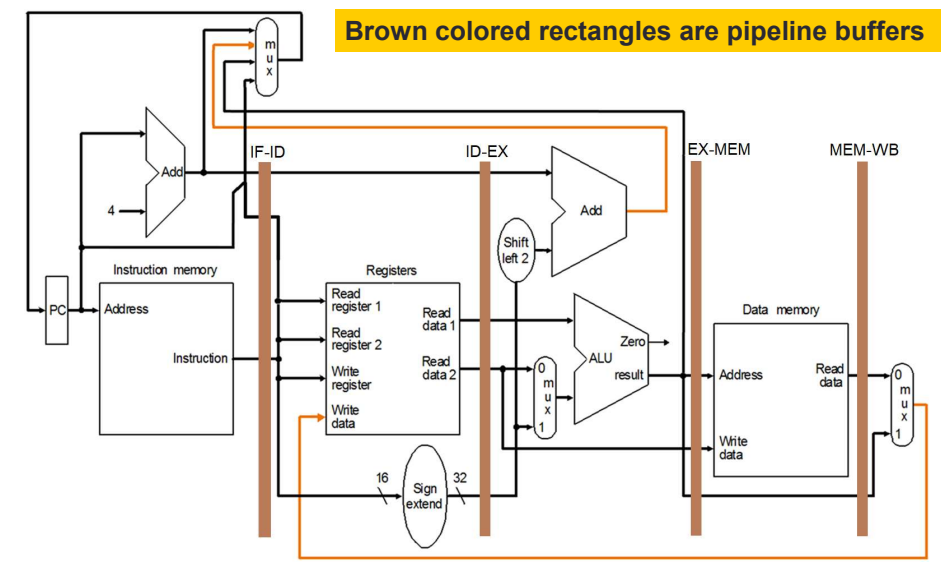
MEM: Memory access

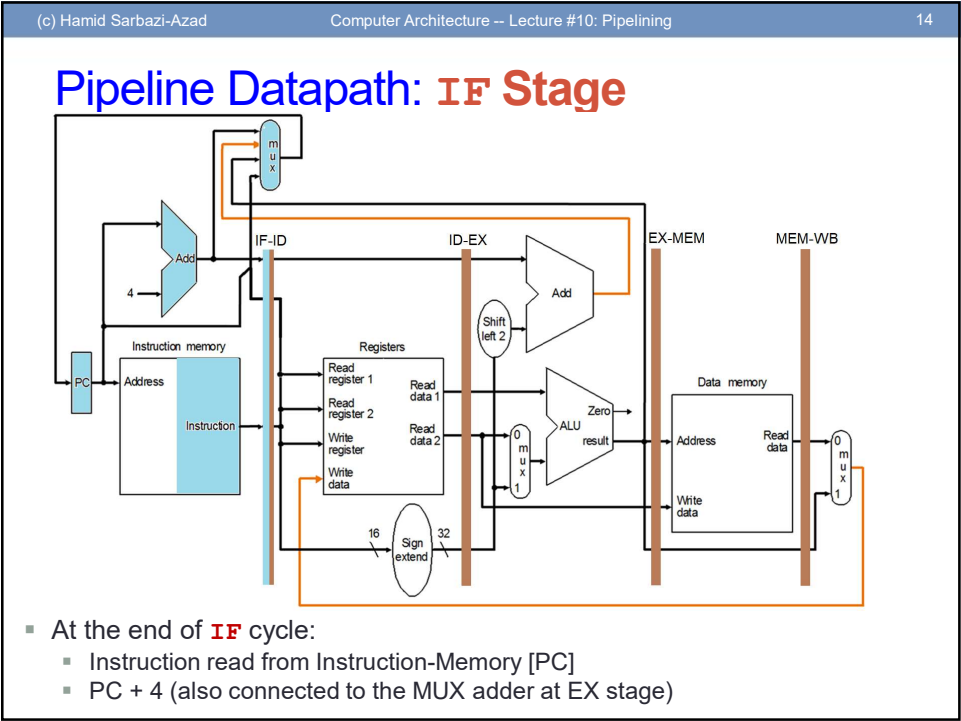
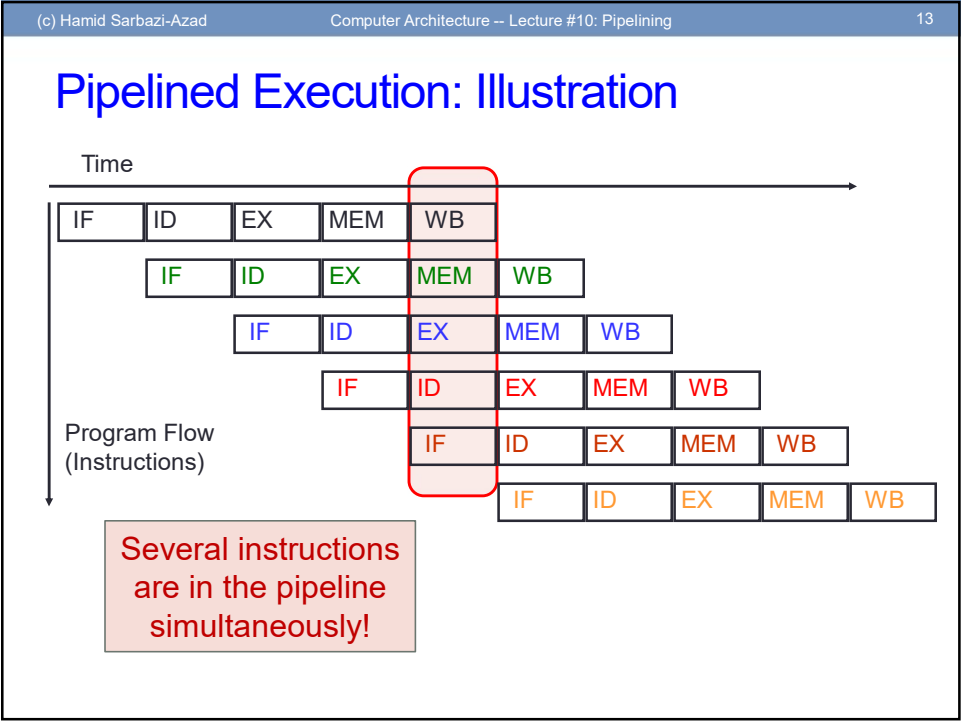
WB: Write back



MIPS Pipeline

Brown colored rectangles are pipeline buffers



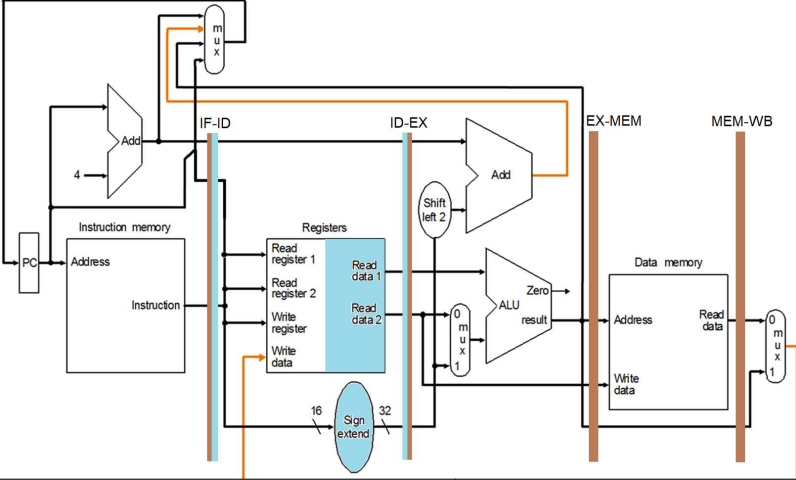


(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

15

Pipeline Datapath: ID Stage



At the beginning of ID stage:

At the end of ID stage:

- ❖ Register numbers for reading two registers
- ❖ 16-bit offset to be sign-extended to 32-bit

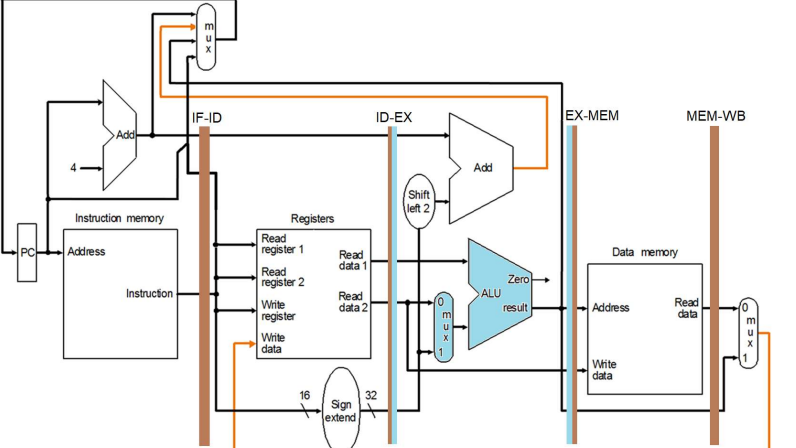
- ❖ Data values read from register file
- ❖ 32-bit immediate value
- ❖ PC + 4 for EX stage

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

16

Pipeline Datapath: EX Stage



At the beginning of EX stage:

At the end of EX stage:

- ❖ Data values read from register file
- ❖ 32-bit immediate value
- ❖ PC + 4

- ❖ (PC + 4) + (Immediate x 4)
- ❖ ALU result
- ❖ isZero? signal
- ❖ Data Read 2 from register file

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

17

Pipeline Datapath: MEM Stage

At the beginning MEM stage:

- ❖ $(PC + 4) + (Immediate \times 4)$
- ❖ ALU result
- ❖ `isZero?` signal
- ❖ Data Read 2 from register file

At the end of MEM stage:

- ❖ ALU result
- ❖ Memory read data

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

18

Pipeline Datapath: WB Stage

At the beginning of WB stage:

- ❖ ALU result
- ❖ Memory read data

At the end of WB stage:

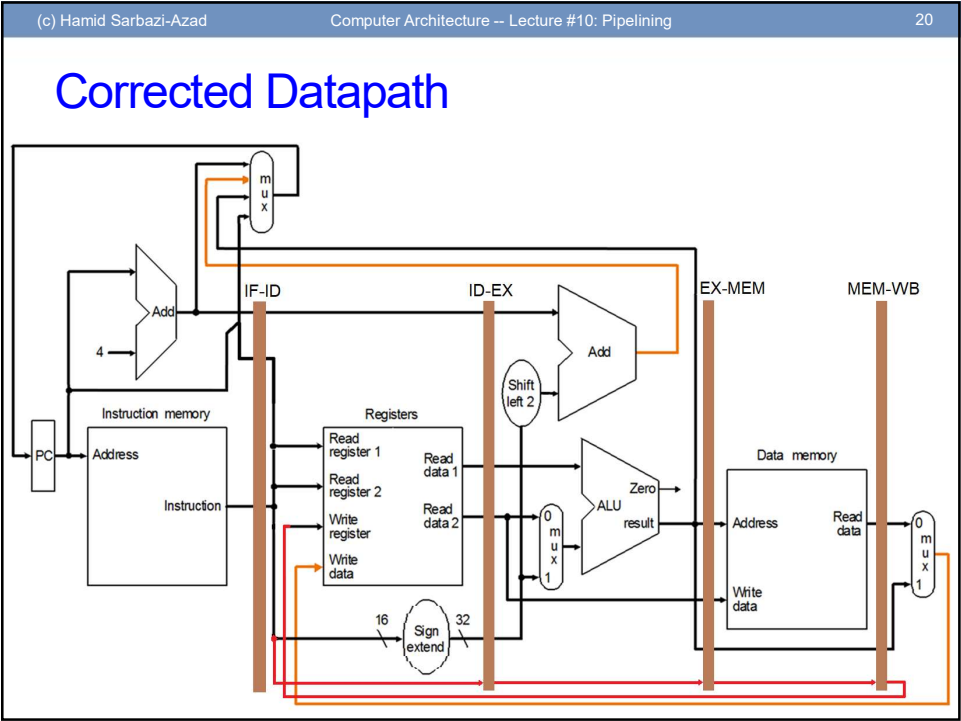
- ❖ Result is written back to register file (if applicable)
- ❖ **There is a bug here.....**

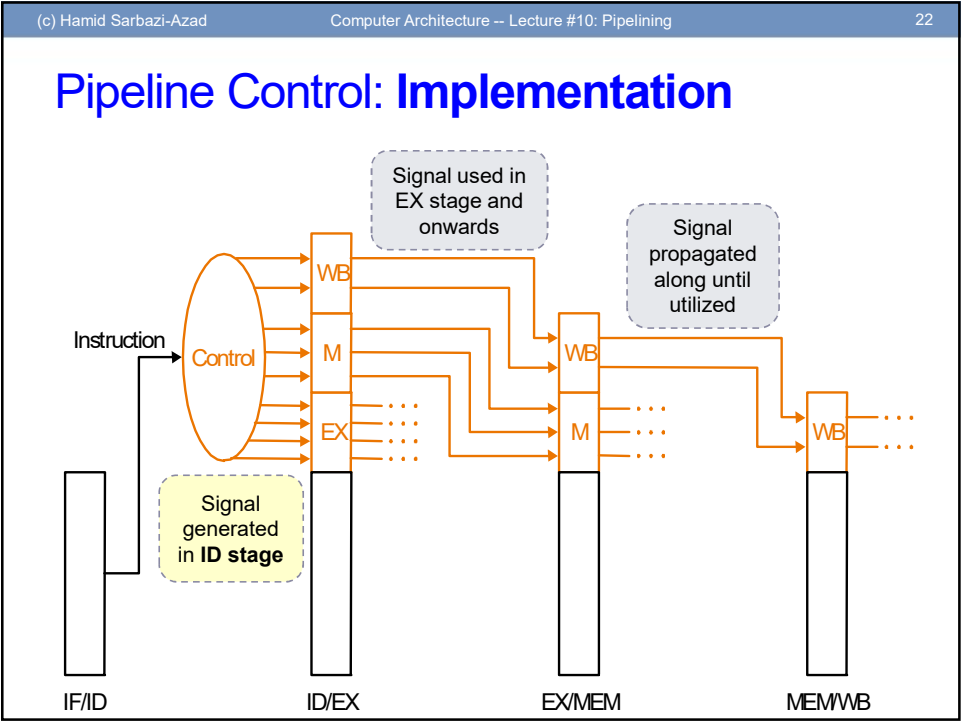
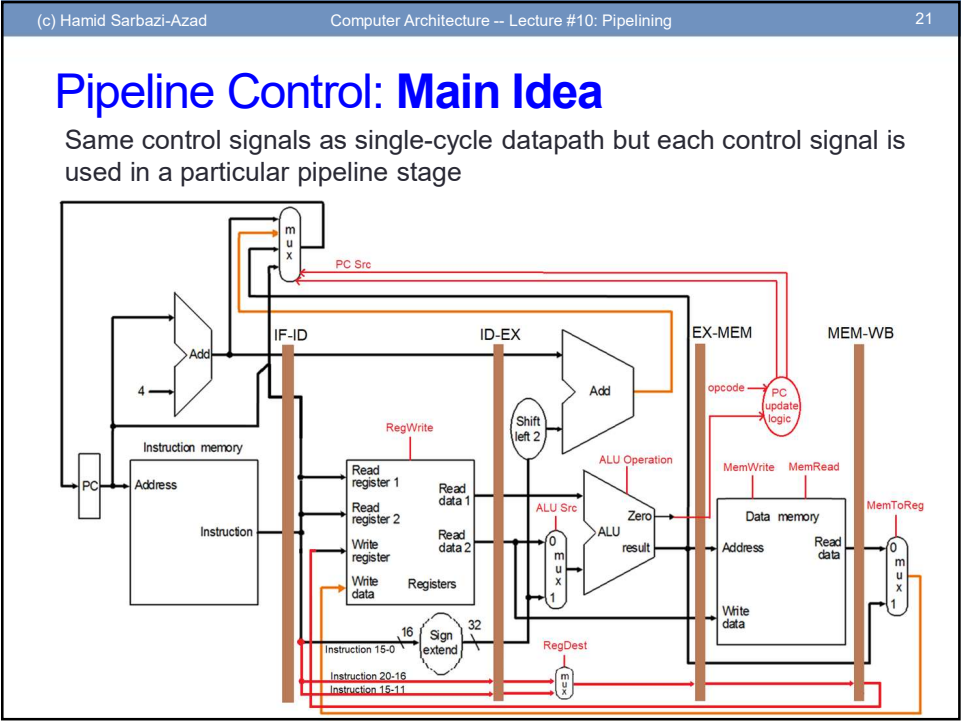
(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #10: Pipelining19

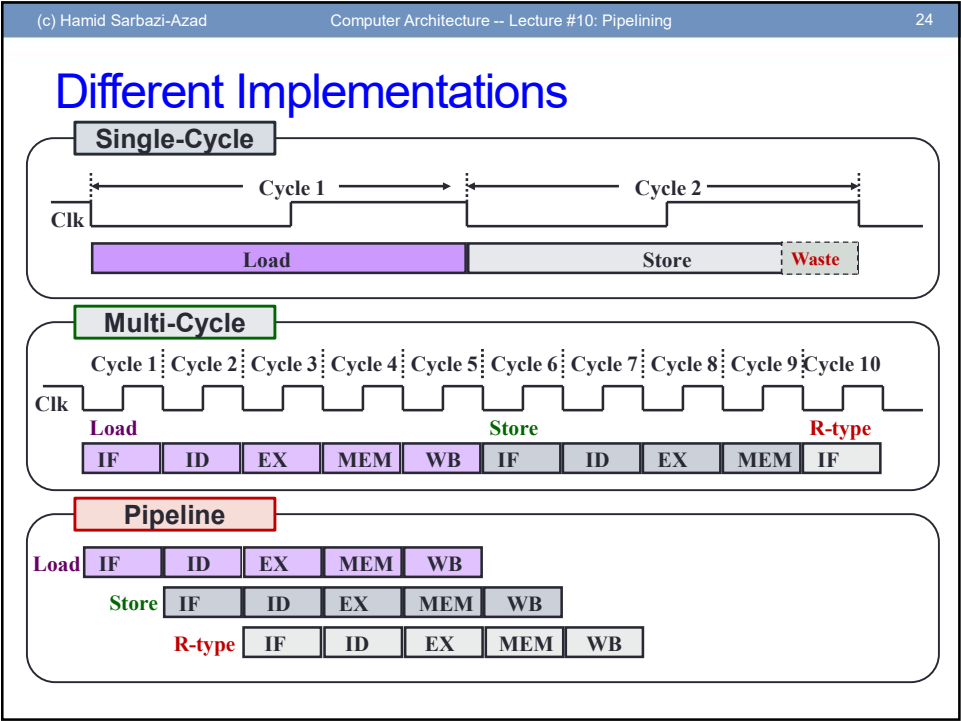
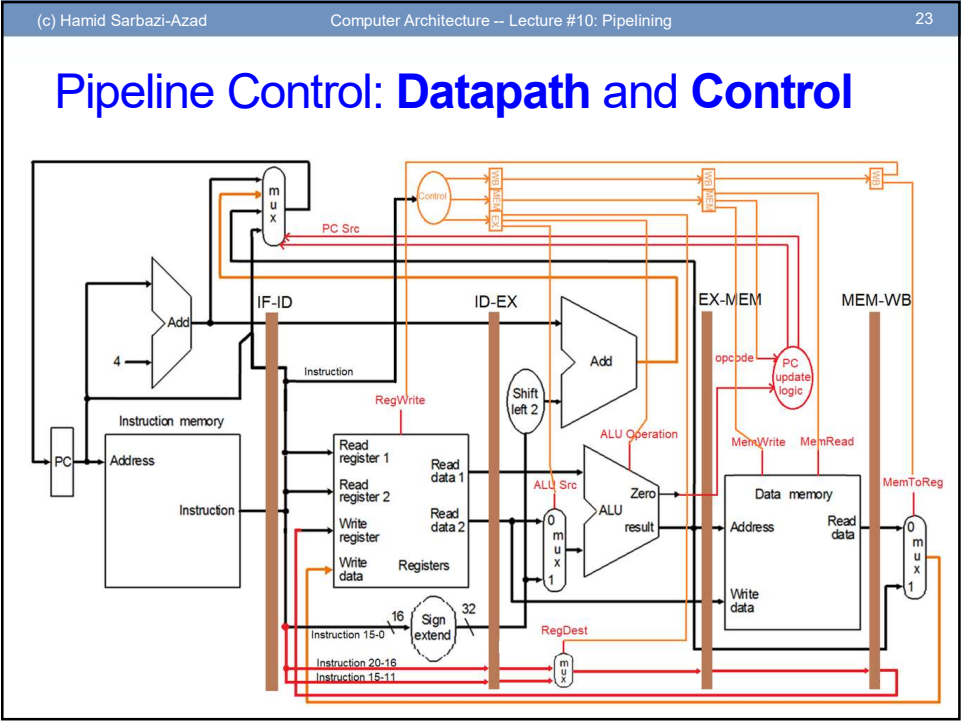
Corrected Datapath

- Observe the “Write register” number
 - Supplied by the **IF/ID** pipeline register
 - It is NOT the correct write register for the instruction now in **WB** stage!
- Solution:**
 - Pass “Write register” number from **ID** through **EX** and **MEM** to **WB** stage for use in **WB** stage

i.e. let the "Write register" number follows the instruction through the pipeline until it is needed in WB stage







(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #10: Pipelining25

Single Cycle Processor: Performance

- Cycle time:
 - $CT_{seq} = \sum_{k=1}^n T_k$
 - T_k = Time for operation in stage k
 - n = Number of stages
- Total Execution Time for **N** instructions:
 - $Time_{seq} = Cycles \times CycleTime$
 $= N \times CT_{seq} = N \times \sum_{k=1}^n T_k$

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #10: Pipelining26

Single Cycle Processor: Example

Instruction	Inst Mem	Reg read	ALU	Data Mem	Reg write	Total
ALU	2 ns	1 ns	2 ns		1 ns	6 ns
lw	2 ns	1 ns	2 ns	2 ns	1 ns	8 ns
sw	2 ns	1 ns	2 ns	2 ns		7 ns
beq	2 ns	1 ns	2 ns			5 ns

- **Cycle Time**
 - Choose the longest total time = **8ns**
- To execute **100 instructions**:
 - **100 × 8ns = 800ns**

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

27

Multi-Cycle Processor: Performance

- Cycle time:
 - $CT_{multi} = \max(T_k)$
 - $\max(T_k)$ = longest stage duration among the n stages
- Total Execution Time for **N** instructions:
 - $Time_{multi} = Cycles \times CycleTime$
 $= N \times \textit{Average CPI} \times CT_{multi}$
 - Average CPI is needed because each instruction takes different number of cycles to finish

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

28

Multi-Cycle Processor: Example

Instruction	Inst Mem	Reg read	ALU	Data Mem	Reg write	Total
ALU	2 ns	1 ns	2 ns		1 ns	6 ns
lw	2 ns	1 ns	2 ns	2 ns	1 ns	8 ns
sw	2 ns	1 ns	2 ns	2 ns		7 ns
beq	2 ns	1 ns	2 ns			5 ns

- **Cycle Time**
 - Choose the longest stage time = **2ns**
- To execute **100 instructions**, with a given **average CPI of 4.2**
 - $100 \times 4.2 \times 2ns = 840ns$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

29

Pipeline Processor: Performance

- Cycle Time:
 - $CT_{pipeline} = \max(T_k) + T_d$
 - $\max(T_k)$ = longest time among the n stages
 - T_d = Overhead for pipelining, e.g. pipeline register
- Cycles needed for **N** instructions:
 - $n + N - 1$
 - n is the cycles wasted in filling up the pipeline
- Total Time needed for **N** instructions :
 - $Time_{pipeline} = Cycle \times CT_{pipeline}$
 $= (N + n - 1) \times (\max(T_k) + T_d)$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

30

Pipeline Processor: Example

Instruction	Inst Mem	Reg read	ALU	Data Mem	Reg write	Total
ALU	2 ns	1 ns	2 ns		1 ns	6 ns
lw	2 ns	1 ns	2 ns	2 ns	1 ns	8 ns
sw	2 ns	1 ns	2 ns	2 ns		7 ns
beq	2 ns	1 ns	2 ns			5 ns

- Cycle Time
 - assume pipeline register delay of **0.5ns**
 - longest stage time + overhead = **2 + 0.5 = 2.5ns**
- To execute **100 instructions**:
 - **$(100 + 5 - 1) \times 2.5ns = 260ns$**

Pipeline Processor: Ideal Speedup

- Assumptions for ideal case:

- Every stage takes the same amount of time:

$$\rightarrow \sum_{k=1}^n T_k = n \times T_k$$

- No pipeline overhead $\rightarrow T_d = 0$

Number of instructions N is much larger than number of stages n

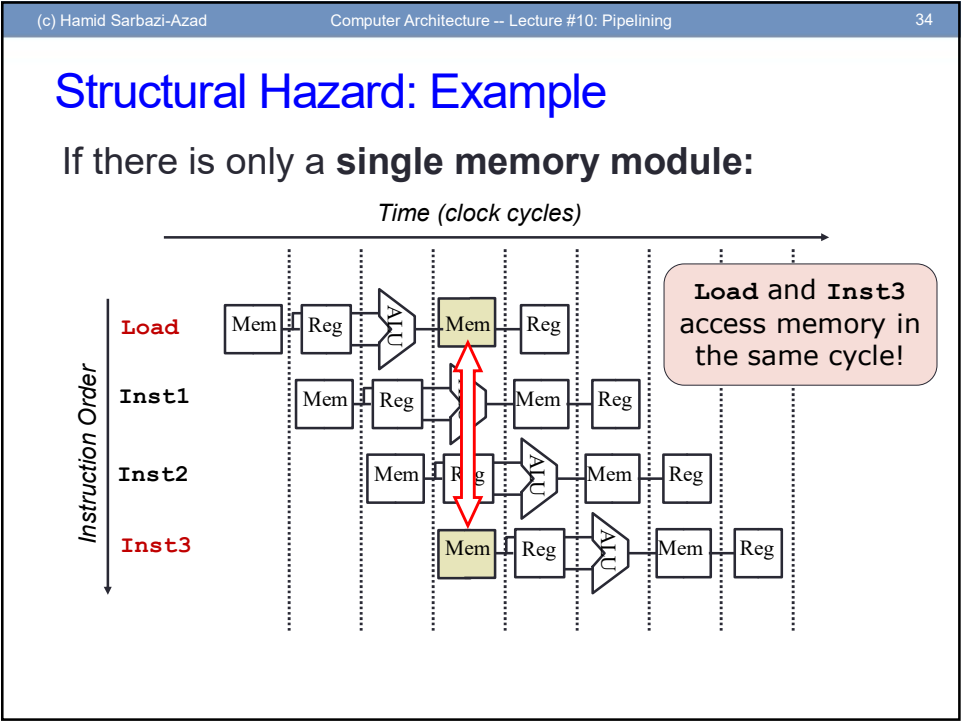
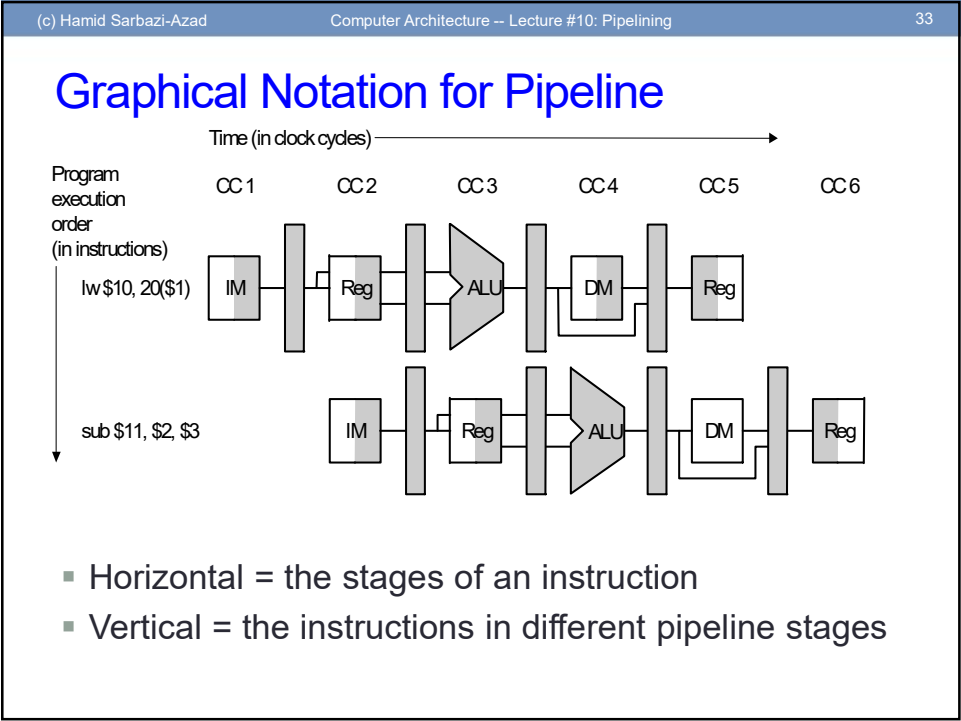
Conclusion:

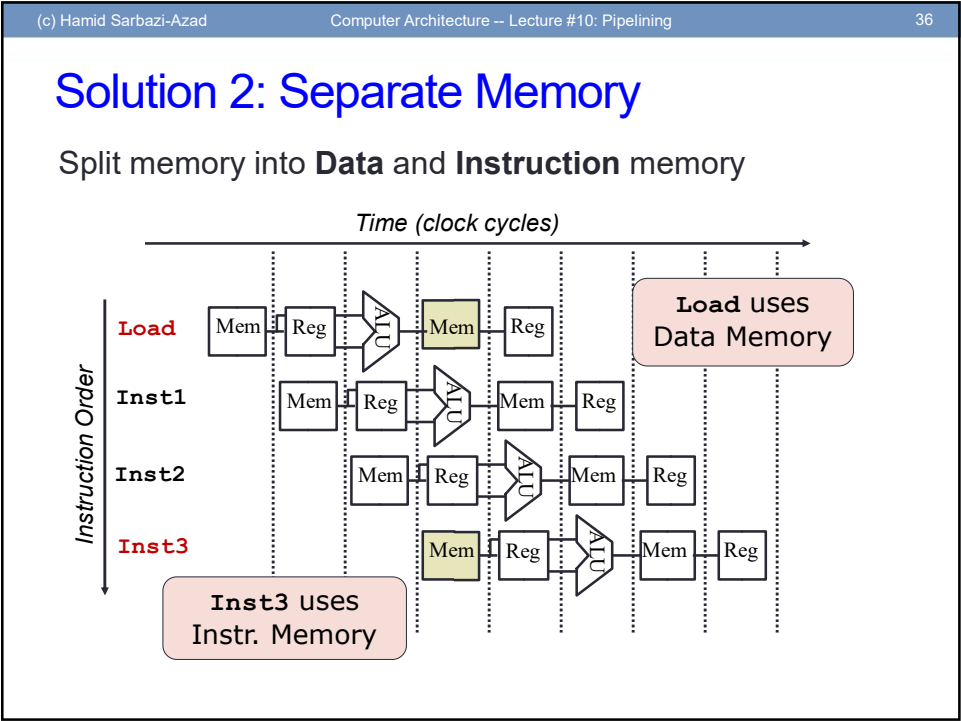
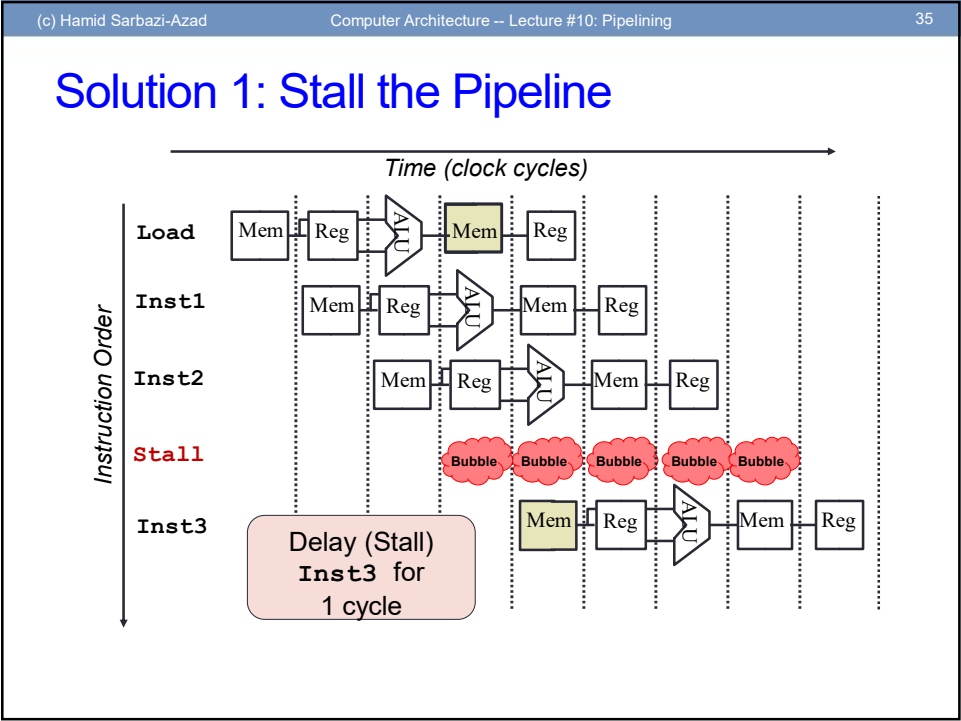
Pipeline processor can gain n times speedup, where n is the number of pipeline stages if we can keep the pipeline full

$$\approx \frac{n \times N \times T_k}{N \times T_k} \approx n \quad \text{for large values of } N$$

Pipeline Hazards

- Speedup n is gained if new instructions can be "pumped" into pipeline every cycle
- However, there are problems that prevent next instruction from immediately following previous instruction.
- They are:
 - Structural hazards**
 - Simultaneous use of a hardware resource
 - Data hazards**
 - Data dependencies between instructions
 - Control hazards**
 - Change in program flow





Instruction Dependencies

Instructions can have relationship that prevent pipeline execution:

- When different instructions accesses (read/write) the same register
 - Register contention! (aka **data dependency**)
- When the execution of an instruction depends on another instruction
 - Control flow is the cause of dependency (aka **control dependency**)

Failure to properly handle dependencies can affect **program correctness**!

Data Dependency: RAW

- "Read-After-Write" Definition:
 - Occurs when a later instruction **reads** from the destination register **written** by an earlier instruction
- Also known as **true data dependency**

```
i1: add $1, $2, $3 #writes to $1
i2: sub $4, $1, $5 #reads from $1
```

Effect of incorrect execution:

- If i2 reads register \$1 before i1 can write back the result, i2 will get a **stale result** (old result)

Other Data Dependencies

- Similarly, we have:
 - **WAR: Write-after-Read** dependency
 - **WAW: Write-after-Write** dependency
- Fortunately, these dependencies do not cause any pipeline hazards in MIPS due to its specific stages and ISA
- They affect the processor only when instructions are executed out of program order:
 - It can happen in modern Superscalar processors

RAW Dependency: Hazards?

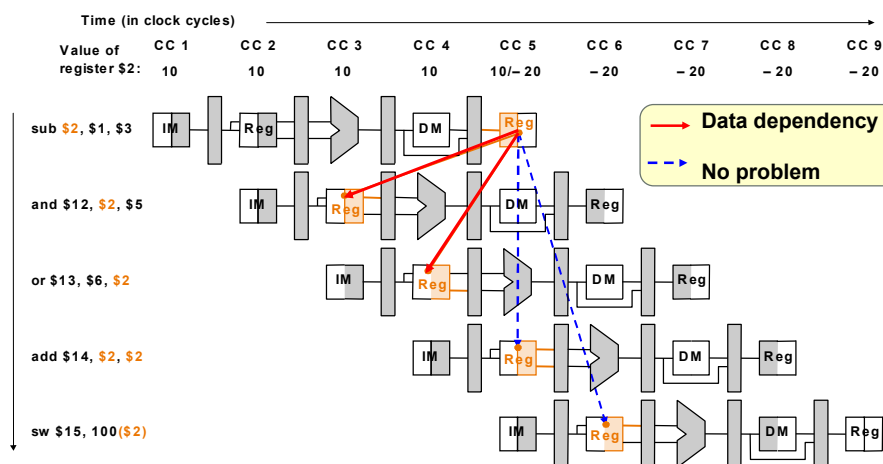
- Suppose we are executing the following code fragment:

```
sub $2, $1, $3    #i1
and $12, $2, $5   #i2
or  $13, $6, $2   #i3
add $14, $2, $2   #i4
sw  $15, 100($2)  #i5
```

- Note the multiple uses of register **\$2**

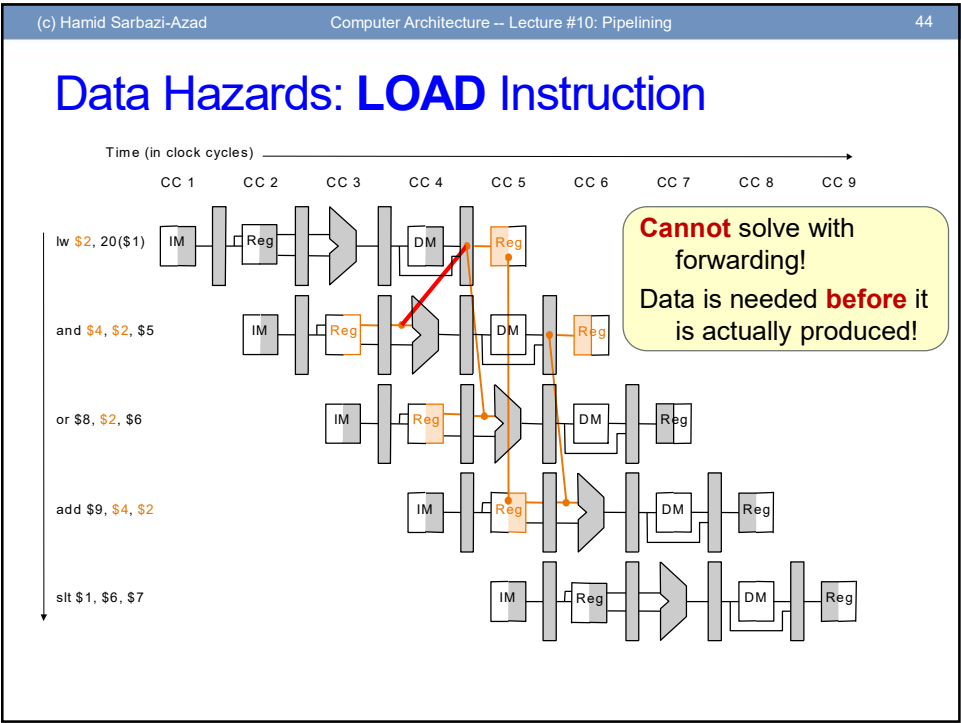
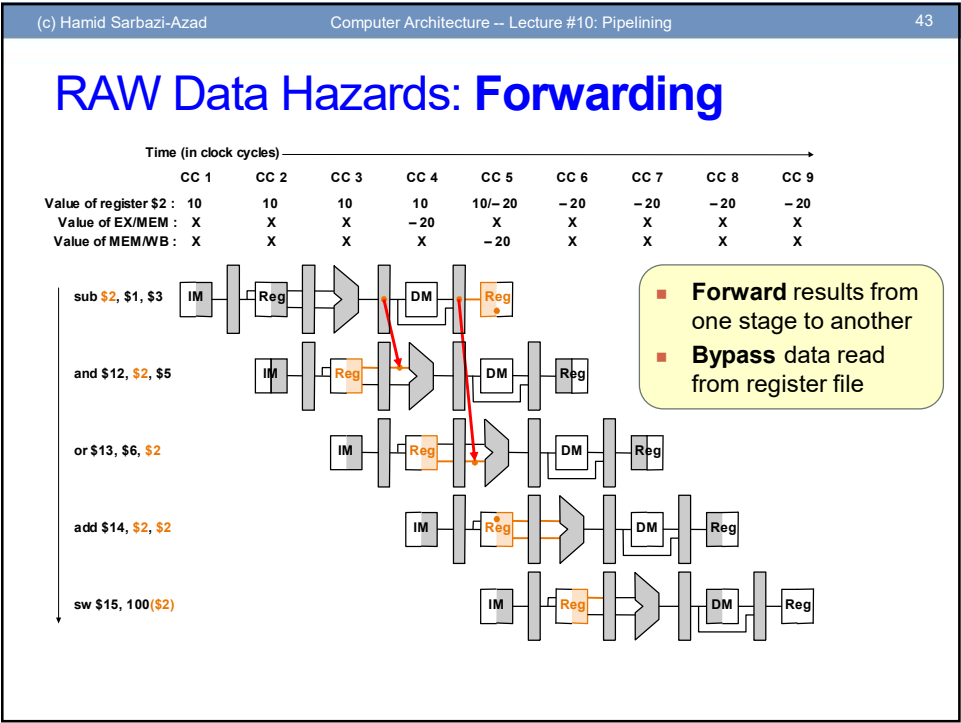
RAW Data Hazards

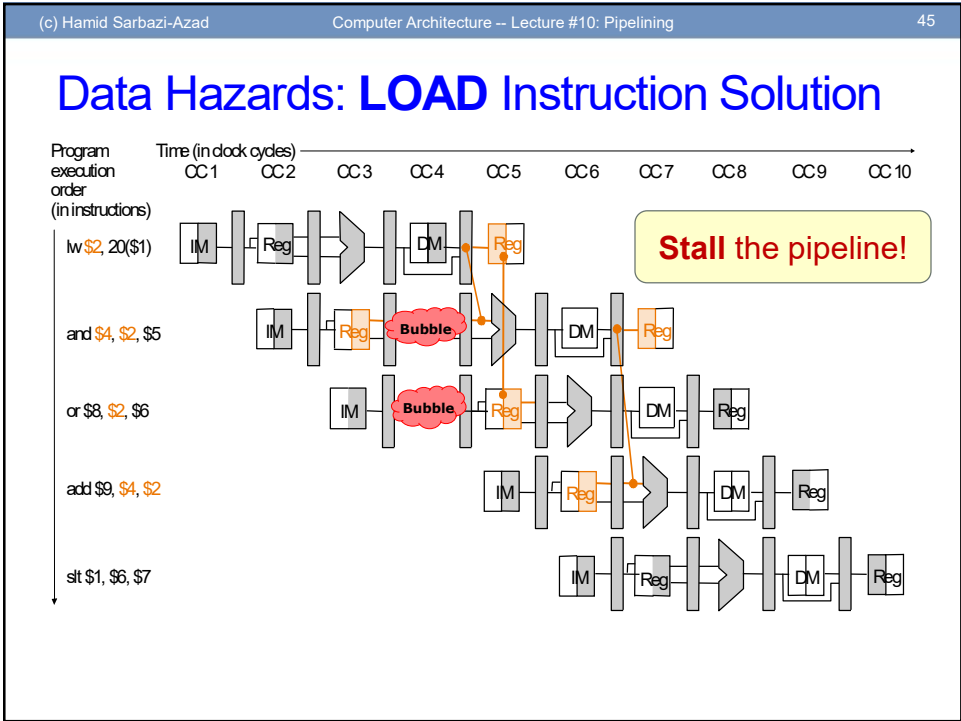
- Value from prior instruction is needed before write back



RAW Data Hazards: Observations

- Questions:
 - When is the result from **sub** instruction actually produced?
 - End of EX stage for **sub** or clock cycle 3
 - When is the data actually needed by **and**?
 - Beginning of **and**'s EX stage or clock cycle 4
 - When is the data actually needed by **or**?
 - Beginning of **or**'s EX stage or clock cycle 5
- Solution:
 - Forward** the result to any trailing (later) instructions before it is reflected in register file
 - **Bypass (replace)** the data read from register file





(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

46

Example 1:

- How many cycles will it take to execute the following code on a 5-stage pipeline
 - without** forwarding?
 - with** forwarding?

```
sub $2, $1, $3
and $12, $2, $5
or $13, $6, $2
add $14, $2, $2
sw $15, 100($2)
```

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

47

Example 1: Without Forwarding

sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)

	1	2	3	4	5	6	7	8	9	10	11
sub	IF	ID	EX	MEM	WB						
and		IF			ID	EX	MEM	WB			
or					IF	ID	EX	MEM	WB		
add						IF	ID	EX	MEM	WB	
sw							IF	ID	EX	MEM	WB

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

48

Example 1: With Forwarding

sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)

	1	2	3	4	5	6	7	8	9	10	11
sub	IF	ID	EX	MEM	WB						
and		IF	ID	EX	MEM	WB					
or			IF	ID	EX	MEM	WB				
add				IF	ID	EX	MEM	WB			
sw					IF	ID	EX	MEM	WB		

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #10: Pipelining49

Example 2:

- How many cycles will it take to execute the following code on a 5-stage pipeline
 - without** forwarding?
 - with** forwarding?

```
lw  $2, 20($3)
and $12, $2, $5
or  $13, $6, $2
add $14, $2, $2
sw  $15, 100($2)
```

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #10: Pipelining50

Example 2: Without Forwarding

```
lw  $2, 20($3)
and $12, $2, $5
or  $13, $6, $2
add $14, $2, $2
sw  $15, 100($2)
```

	1	2	3	4	5	6	7	8	9	10	11
lw	IF	ID	EX	MEM	WB						
and		IF			ID	EX	MEM	WB			
or					IF	ID	EX	MEM	WB		
add						IF	ID	EX	MEM	WB	
sw							IF	ID	EX	MEM	WB

◇

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

51

Example 2: With Forwarding

lw \$2, 20(\$3)

and \$12, \$2, \$5

or \$13, \$6, \$2

add \$14, \$2, \$2

sw \$15, 100(\$2)

	1	2	3	4	5	6	7	8	9	10	11
lw	IF	ID	EX	MEM	WB						
and		IF	ID		EX	MEM	WB				
or			IF		ID	EX	MEM	WB			
add					IF	ID	EX	MEM	WB		
sw						IF	ID	EX	MEM	WB	

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

52

Control Dependency

Definition:

An instruction *j* is control dependent on *i* if *i* controls whether or not *j* executes

Typically *i* would be a branch instruction

Example:

i1: beq \$3, \$5, label # branch

i2: add \$1, \$2, \$4 # depends on i1

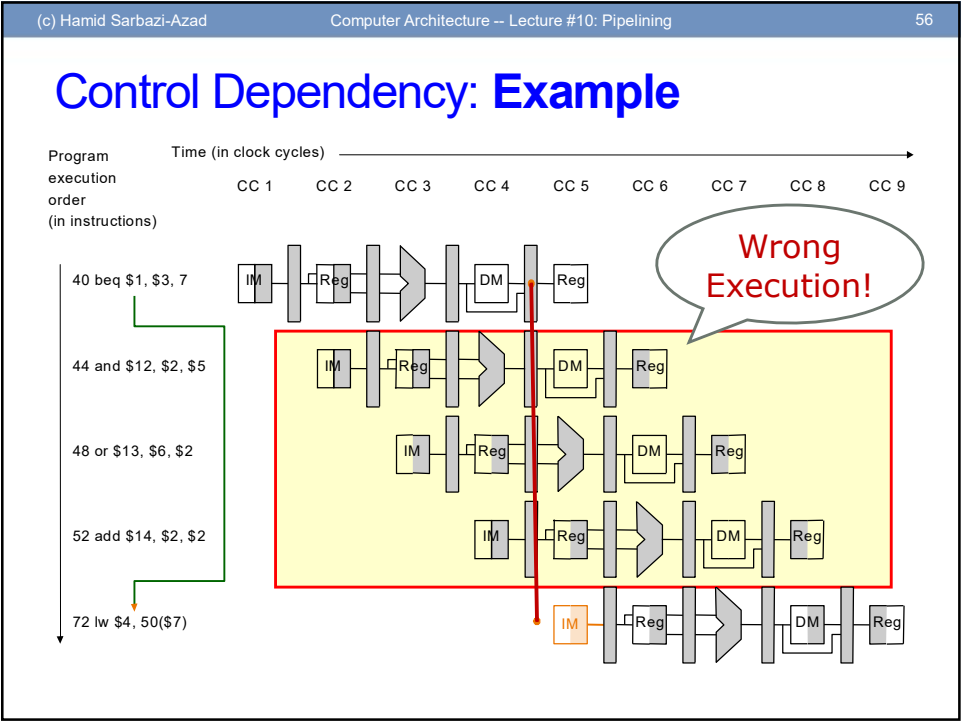
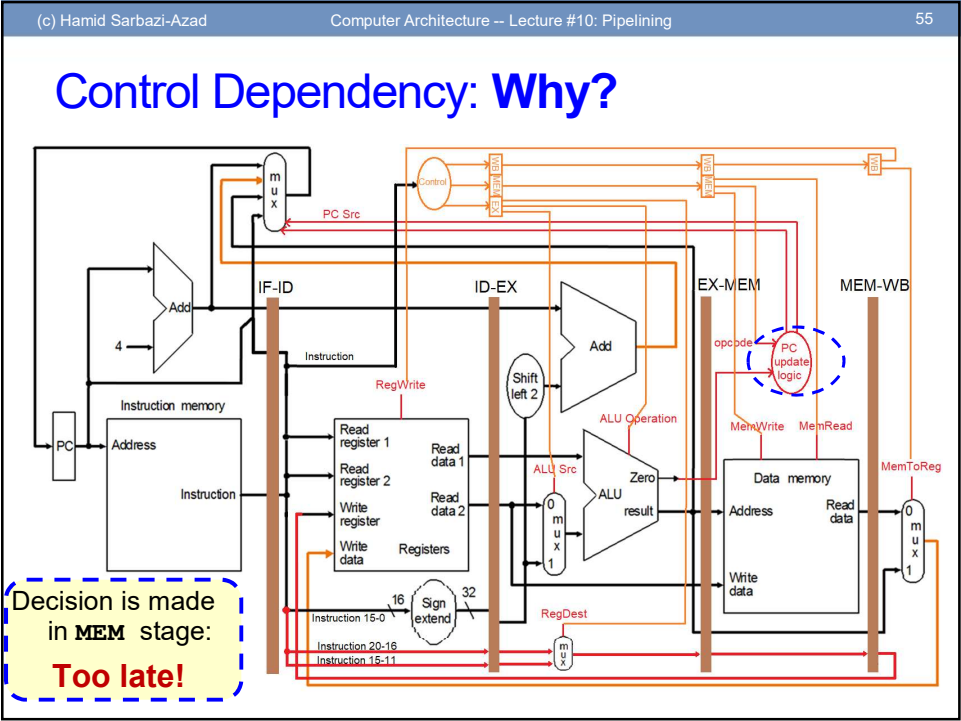
...

Effect of incorrect execution:

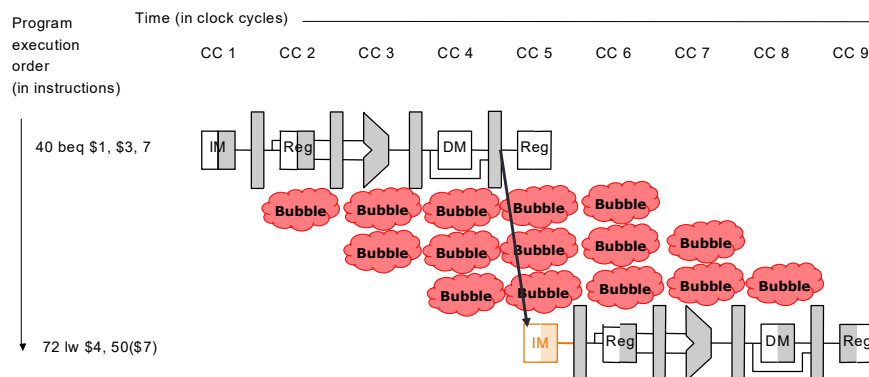
If *i2* is allowed to execute before *i1* is determined, register \$1 maybe incorrectly changed!

26

27



Control Hazards: Stall Pipeline?



- Wait until the branch outcome is known and then fetch the correct instructions
- ➔ Introduces **3 clock stall cycles**

Control Hazards: Reducing the Penalty

- Branching is very common in code:
 - A 3-cycle stall penalty is too heavy!
- Many techniques invented to reduce the control hazard penalty:
 - Move branch decision calculation to earlier pipeline stage
 - **Early Branch Resolution**
 - Guess the outcome before it is produced
 - **Branch Prediction**
 - Do something useful while waiting for the outcome
 - **Delayed Branching**

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

59

Reduce Stalls: Early Branch

▪ Make decision in **ID** stage instead of **MEM**

▪ Move branch target address calculation

▪ Move register comparison → cannot use ALU for register comparison any more

Branch target address calculation

PC + 4

Shift left 2

Add

Target address

Instruction

Read register 1

Read register 2

Write register

Write data

RegWrite

Read data 1

Read data 2

Sign extend

16

32

4

ALUcontrol

Zero

ALU ALU result

To branch Control logic

Register Comparison

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

60

Reduce Stalls: Early Branch

Register comparison moved to ID stage

PC

Instruction memory

Address

Instruction

IF-ID

Read register 1

Read register 2

Write register

Write data

Read data 1

Read data 2

Sign extend

16

32

PC Src

PC update logic

RegWrite

Zero

Shift left 2

Add

4

ALU Src

0

1

ALU result

ALU Operation

RegDest

mux

EX-MEM

MemWrite

MemRead

MemToReg

MEM-WB

0

1

mux

30

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

61

Reduce Stalls: Early Branch

Program execution order (in instructions)

Time (in clock cycles)

CC 1 CC 2 CC 3 CC 4 CC 5 CC 6 CC 7 CC 8 CC 9

40 beq \$1, \$3, 7

72 lw \$4, 50(\$7)

- Wait until the branch decision is known:
 - Then fetch the correct instruction
- Reduced from 3 to **1 clock cycle delay**

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

62

Early Branch: Problems

However, if the register(s) involved in the comparison is produced by preceding instruction:

- Further stall is still needed!

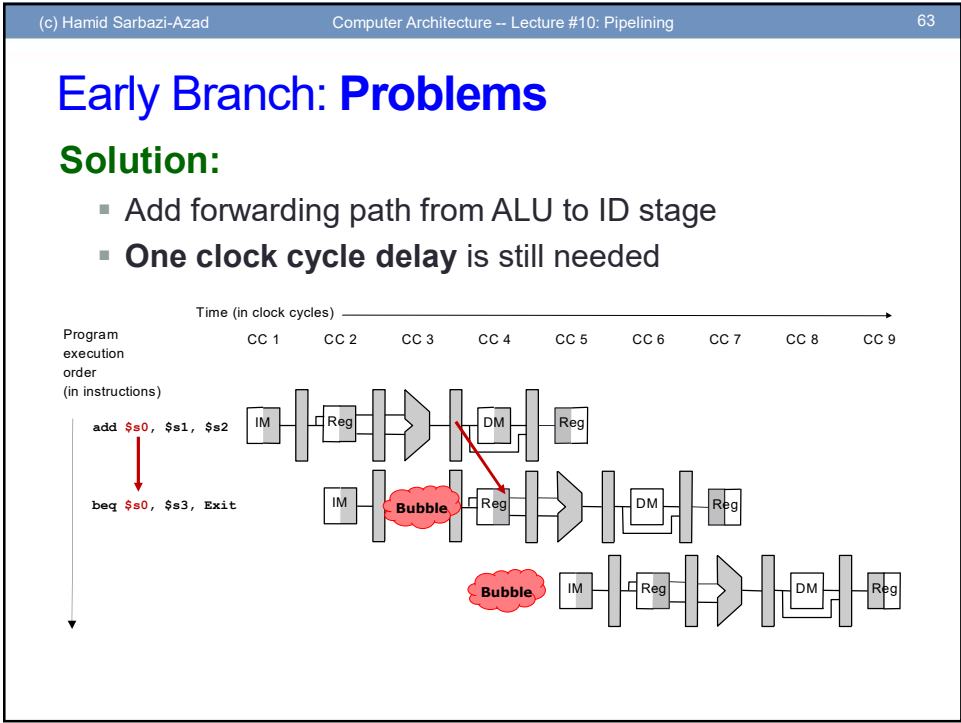
Time (in clock cycles)

CC 1 CC 2 CC 3 CC 4 CC 5 CC 6 CC 7 CC 8 CC 9

add \$s0, \$s1, \$s2

beq \$s0, \$s3, Exit

\$s0 is needed before it is produced!



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

65

Reduce Stalls: Branch Prediction

There are many branch prediction schemes

- Static prediction v.s. Dynamic prediction

Example: A simple static prediction: follow the PC!

- i.e. all branches are assumed to be **not taken!**

→ Fetch the successor instruction and start pumping it through the pipeline stages

When the actual branch outcome is known:

- **Not taken**: Guessed correctly → No pipeline stall
- **Taken**: Guessed wrongly
 - Wrong instructions in the pipeline
 - **Flush** successor instruction from the pipeline

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

66

Branch Prediction: Correct Prediction

Time (in clock cycles) →

Program execution order (in instructions)

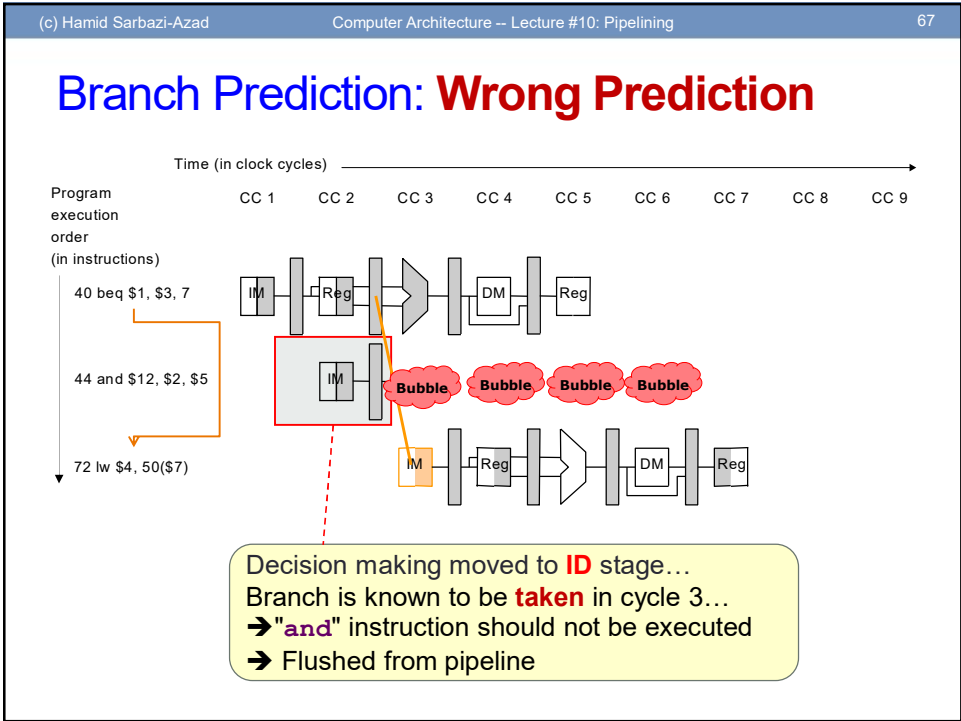
40 beq \$1, \$3, 7

44 and \$12, \$2, \$5

48 or \$13, \$6, \$2

52 add \$14, \$2, \$2

Branch is known to be **not taken** in cycle 3 → no stall needed!



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

68

Example 3: Branch Prediction

How many cycles will it take to execute the following code on a 5-stage pipeline **with** forwarding and ...

▪ **without** branch prediction?

▪ **with** branch prediction (predict not taken)?

addi \$s0, \$zero, 10

Loop: addi \$s0, \$s0, -1

bne \$s0, \$zero, Loop

sub \$t0, \$t1, \$t2

Decision making moved to **ID** stage.

Total #instructions = 1 + 10×2 + 1 = 22

Ideal pipeline time = 4 + 22 = 26 cycles

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

69

Example 3: without Branch Prediction

	1	2	3	4	5	6	7	8	9	10	11
addi ¹	IF	ID	EX	MEM	WB						
addi ²		IF	ID	EX	MEM	WB					
bne			IF		ID	EX	MEM	WB			
addi ²						IF	ID	EX	MEM	WB	

- Data dependency between (addi \$s0, \$s0, -1) and bne incurs 1 cycle of delay. There are 10 iterations, hence 10 cycles of delay.
- Every bne incurs a cycle of delay to execute the next instruction. There are 10 iterations, hence 10 cycles of delay.
- Total number of cycles of delay = 20.
- Total execution cycles = 26 + 20 = 46 cycles.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #10: Pipelining

70

Example 3: with Branch Prediction

	1	2	3	4	5	6	7	8	9	10	11
addi ¹	IF	ID	EX	MEM	WB						
addi ²		IF	ID	EX	MEM	WB					
bne			IF		ID	EX	MEM	WB			
sub					IF						
addi ²						IF	ID	EX	MEM	WB	

Predict not taken.

- The data dependency remains, hence 10 cycles of delay for 10 iterations.
- In the first 9 iterations, the branch prediction is wrong, hence 1 cycle of delay.
- In the last iteration, the branch prediction is correct, hence saving 1 cycle of delay.
- Total number of cycles of delay = 19.
- Total execution cycles = 26 + 19 = 45 cycles.

Reduce Stalls: Delayed Branch

- **Observation:**
 - Branch outcome takes **X** number of cycles to be known
 - **X** cycles stall
- **Idea:**
 - Move **non-control dependent instructions** into the X slots following a branch
 - Known as the **branch-delay slot**
 - These instructions are executed **regardless of the branch outcome**
- In our MIPS processor:
 - Branch-Delay slot = **1** (with the early branch)

Delayed Branch: Example

Non-delayed branch

```

or $8, $9, $10
add $1, $2, $3
sub $4, $5, $6
beq $1, $4, Exit
xor $10, $1, $11

```

Exit:

Delayed branch

```

add $1, $2, $3
sub $4, $5, $6
beq $1, $4, Exit
or $8, $9, $10
xor $10, $1, $11

```

Exit:

- The "or" instruction is moved into the delayed slot:
 - Get executed regardless of the branch outcome
 - Same behavior as the original code!

Delayed Branch: Observation

- **Best case scenario**

- There is an instruction **preceding the branch** which **can be moved** into the delayed slot
 - Program correctness must be preserved!

- **Worst case scenario**

- Such instruction cannot be found
 - ➔ Add a no-op (**nop**) instruction in the branch-delay slot

- Re-ordering instructions is a common method of program optimization

- Compiler must be smart enough to do this
- Usually can find such an instruction at least 50% of the time

END OF SLIDES

QUESTIONS?